

Módulo 3 — Model Development Support (12h)

Objetivos do Módulo

Ao final deste módulo, os formandos deverão ser capazes de:

- Compreender o papel do desenvolvimento de modelos no ciclo de vida de um projeto de Ciência de Dados.
- Apoiar na criação de modelos supervisionados e não supervisionados.
- Realizar *feature engineering* para melhorar a performance dos modelos.
- Selecionar algoritmos adequados ao tipo de problema.
- Avaliar modelos com métricas apropriadas e técnicas de validação.
- Documentar e comunicar o processo de modelação.

3.2 Conteúdos Desenvolvidos

Conteúdos

- Fluxo de trabalho preditivo: preparação, treino, validação, teste e implementação.
- Preparação dos dados: normalização, codificação, divisão treino/teste, imputação.
- Engenharia de variáveis: criação, transformação, seleção, PCA.
- Treino de modelos: `caret`, `tidymodels`, `recipes`.
- Algoritmos: regressão, classificação, árvores, random forest, k-NN, SVM.
- Avaliação: RMSE, MAE, R^2 , Acurácia, F1, AUC, validação cruzada.
- Comparação e seleção de modelos.

3.3 Atividades Práticas Detalhadas

Atividades Práticas

- Criar um modelo de regressão para prever preços de imóveis com `caret`.
- Desenvolver um modelo de classificação para prever *churn* com `tidymodels`.
- Implementar um *recipe* com normalização e codificação.
- Comparar dois algoritmos diferentes para o mesmo problema.
- Documentar o processo e apresentar métricas e gráficos.

3.4 Recursos e Ferramentas

Ferramentas

- Software: R e RStudio.
- Pacotes: caret, tidymodels, recipes, ggplot2, dplyr.
- Datasets: preços de imóveis, churn, UCI Repository.

3.5 Estudo de Caso — Desenvolvimento de Modelo em R

Estudo de Caso

Objetivo: Demonstrar todas as etapas do desenvolvimento de um modelo preditivo.

```
1 library(tidyverse)
2 library(caret)
3 library(tidymodels)
4
5 # 1. Importar dataset
6 dados <- read_csv("precos_imoveis.csv")
7
8 # 2. Divis o treino / teste
9 set.seed(123)
10 divisao <- initial_split(dados, prop = 0.7)
11 treino <- training(divisao)
12 teste <- testing(divisao)
13
14 # 3. Recipe de pre-processamento
15 receita <- recipe(preco ~ ., data = treino) %>%
16 step_normalize(all_numeric_predictors()) %>%
17 step_dummy(all_nominal_predictors())
18
19 # 4. Modelo: regress o linear
20 modelo_rl <- linear_reg() %>%
21 set_engine("lm")
22
23 # 5. Workflow
24 workflow_rl <- workflow() %>%
25 add_recipe(receita) %>%
26 add_model(modelo_rl)
27
28 # 6. Treinar
29 modelo_treinado <- fit(workflow_rl, data = treino)
30
31 # 7. Avaliar
32 predicoes <- predict(modelo_treinado, teste) %>%
33 bind_cols(teste)
34
35 metrics(predicoes, truth = preco, estimate = .pred)
36
```

```

37 # 8. Comparar com Random Forest
38 modelo_rf <- rand_forest() %>%
39 set_engine("ranger") %>%
40 set_mode("regression")
41
42 workflow_rf <- workflow() %>%
43 add_recipe(receita) %>%
44 add_model(modelo_rf)
45
46 modelo_rf_treinado <- fit(workflow_rf, data = treino)
47
48 predicoes_rf <- predict(modelo_rf_treinado, teste) %>%
49 bind_cols(teste)
50
51 metrics(predicoes_rf, truth = preco, estimate = .pred)

```

Notas para o Formador

Notas

- Incentivar a experimentação com tuning de hiperparâmetros.
- Discutir vantagens e limitações de cada algoritmo.
- Sugerir relatórios automáticos com RMarkdown.
- Mostrar como interpretar métricas e gráficos de diagnóstico.